Anthony Fuca

ajfuca@gmail.com | github.com/ajfuca4 | linkedin.com/in/ajfuca/

Education

University of Waterloo — Waterloo, ON
Bachelor of Computer Science, Major in Artificial Intelligence, Minor in Combinatorics & Optimization — *Graduating 2027*

Wilfrid Laurier University — Waterloo, ON
Bachelor of Business Administration, Major in Finance — *Graduating 2027*

Experience

Full-Stack Software Engineering Intern — Sept. 2026 – Present
Fig Financial — *Toronto, ON*

- Optimized internal lookup workflows in Django Admin by implementing PostgreSQL trigram and B-tree indexes, reducing p99 search latency by 95% and eliminating request timeouts for operations staff
- Replaced 50+ weekly manual DB patches with an automated Django/React remediation tool, reconciling legacy Flinks financial webhooks using PII-redacted audits and zero external API re-calls
- Eliminated full-table COUNT(*) bottlenecks on 2M+ row production tables by engineering a shared N+1 lookahead pagination pattern in Django Admin, preventing database CPU spikes and query timeouts during peak operator workflows

AI Software Engineering Intern — Jan. 2026 – Apr. 2026
TD Bank — *Toronto, ON*

- Engineered a stateful RAG and memory pipeline in Python, FastAPI, and LangChain for 200+ insurance analysts, decoupling reference material and user intent from hierarchical, recency-weighted summaries to cut token usage by 70%
- Developed and presented hands-on context engineering workshops to 110+ engineers and leaders, driving a 300% increase in non-trivial AI engineering workflows within 2 weeks as measured by internal DevOps AI usage tracking tools
- Built a document ingestion and diffing pipeline in Python and Azure OpenAI, normalizing unstructured legacy filetypes to assist manual analyst reconciliation across roughly 10,000 monthly documents saving an estimated $10M in rework costs

Financial Analyst Intern — Jan. 2025 – Apr. 2025
Nicoya Lifesciences — *Kitchener, ON*

- Increased forecasting accuracy by 20% by architecting a dynamic projection model that ingested and normalized 6 years of fragmented historical accounting data
- Built automated data-transformation pipelines in VBA to parse, validate, and standardize raw financial datasets, eliminating manual processing errors and saving 300+ analyst hours annually
- Managed full-cycle order entry and client invoicing, partnering cross-functionally to troubleshoot account discrepancies, enforce strict turnaround standards, and maintain seamless customer operations

Projects

GradeStar | *JavaScript, Node.js, MongoDB, Express.js, HTML/CSS* — Oct. 2026

- Developed a full-stack JavaScript web app that gives students a unified dashboard to build weighted grading schemes, view course contents, track real-time grades, and manage deadlines on a monthly calendar
- Designed REST APIs to create, duplicate, and delete courses, including a deep copy of nested categories and assignments, with bcrypt-hashed passwords and server-side sessions that redirect unauthenticated requests away from every course route
- Modeled courses, weighted categories, and assignments as nested MongoDB documents on each user record, so Mongoose can load a student's full set of courses, weights, and tasks in one query against MongoDB Atlas
- Wrote unit tests for the application's domain logic with Node's built-in test runner, exercising that behavior as pure functions apart from the database connection and the HTTP layer

Runster - Strava Companion | *Kotlin, Jetpack Compose, Supabase, Strava API* — Nov. 2025

- Built a native Android fitness app that turns runs into progression for a digital companion (levels, mood, weekly goals, and achievements) to keep runners motivated, consistent and having fun
- Integrated the Strava API end to end over OAuth 2.0: deep-link authorization callbacks, token exchange, and automatic access-token refresh. Runs sync into a gamification engine that turns distance into XP, levels, streaks, and achievements
- Architected a native Android app in Kotlin and Jetpack Compose using a layered MVVM design with a loosely coupled, interface-driven data layer, enabling isolated unit testing with MockK
- Built LLM-powered motivational messages with the Groq API (Llama 3.1). The prompts are assembled from each user's live stats (goal progress, streaks, the monster's mood) and the call fails gracefully, so a broken AI response never blocks the UI

Technical Skills

Languages: Python, Swift, Kotlin, JavaScript, C/C++, TypeScript, HTML/CSS, C#, Bash, Racket, LaTeX
Frameworks: React, Node.js, FastAPI, LangChain, Jetpack Compose, EJS, Express.js, Django, React Native
AI & Dev Tools: Git, GitHub, Claude Code, Docker, Gemini Enterprise Agents, Cursor, Azure OpenAI, Microsoft Copilot
Backend & Other: MongoDB, PostgreSQL, Supabase, Jira, Power BI, Tableau, Linear, Microsoft Suite, Google Suite